LAB 7: FUZZ TESTING FOR INPUT VALIDATION BUGS

Software Engineering Laboratory (SE-LAB) — Quality Assurance & Robustness Report

Student: Ganesh Aroda C | **SRN: PES1UG24AM102** | **Repository: SE-LAB / lab7** | **Date: November 2026**

1. Objective & Scope

The objective of Lab 7 is to explore **fuzz testing** as a dynamic security and quality assurance technique. By utilizing the **Hypothesis** property-based testing framework, we subject Python functions to hundreds of thousands of random, malformed, boundary, and unexpected inputs to uncover unhandled crashes, logic anti-patterns, and type validation vulnerabilities. Identified defects are subsequently remediated using defensive programming techniques without degrading original functionality.

2. Introduction to Fuzz Testing

Fuzzing (or Fuzz Testing) is an automated dynamic testing technique that feeds semi-random, unexpected, or invalid data into software interfaces to detect unhandled exceptions, memory corruptions, denial-of-service conditions, and security vulnerabilities. Unlike conventional unit tests—which only evaluate scenarios envisioned by the developer—fuzzers operate orthogonally to human bias, consistently discovering edge cases that manual testing misses.

3. The Hypothesis Property-Based Testing Library

Hypothesis is an advanced property-based testing library for Python. Rather than asserting specific input-output mappings, engineers define high-level *properties* (invariants) that must hold true for all valid or invalid inputs. Key capabilities include:

- **Strategies (st.*):** Built-in generators for text, integers, floats, collections, dicts, booleans, and null values.
- **Shrinking:** Automatically simplifies failing test cases to the minimal reproducible counterexample (e.g., s=None or s='').
- **Replay Database:** Caches failing examples in .hypothesis/ to prevent regressions across test runs.

4. Target Application Functions (highway_toll.py / processor.py)

Function	Expected Input Format	Intended Purpose & Output
parse_plate(plate)	String: STATE-DISTRICT-SERIES-NUMBER (e.g., 'KA-05-AB-1234')	Splits registration plate into 3-tuple: (state.upper(), int(district), int(number)).
trip_cost(text)	String: countxrate (e.g., '3x150')	Parses trip count and rate per trip; computes total cost as int(count) * int(rate).
axle_class(axles)	Integer or numeric string representing axle count (e.g., 2, 3, '4')	Classifies vehicle: 'car' (2 axles), 'truck' (3-4 axles), or 'heavy' (≥ 5 axles).

5. Original Application Execution

The original application was executed via python processor.py with standard valid sample inputs. While it functions under ideal conditions, it completely lacks defensive validation against malformed or unexpected data types.

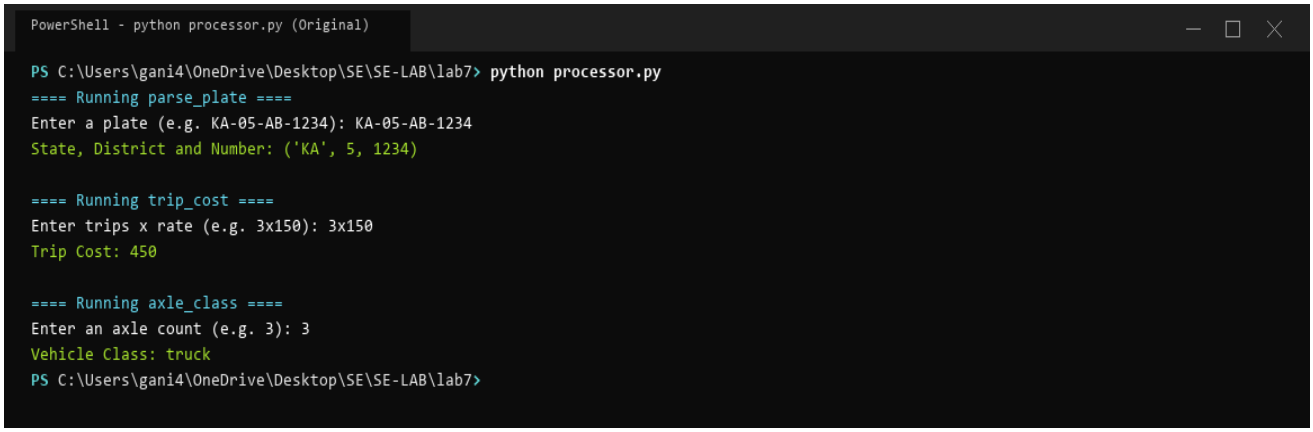


Figure 1: Original application terminal execution with standard inputs.

6. Initial Fuzz Testing Failures

Executing Hypothesis property-based fuzz tests (pytest -v test_processor.py) against the original implementation instantly exposed severe defects. Seven test cases failed due to unhandled crashes on None, empty strings, lists, and negative boundary values.

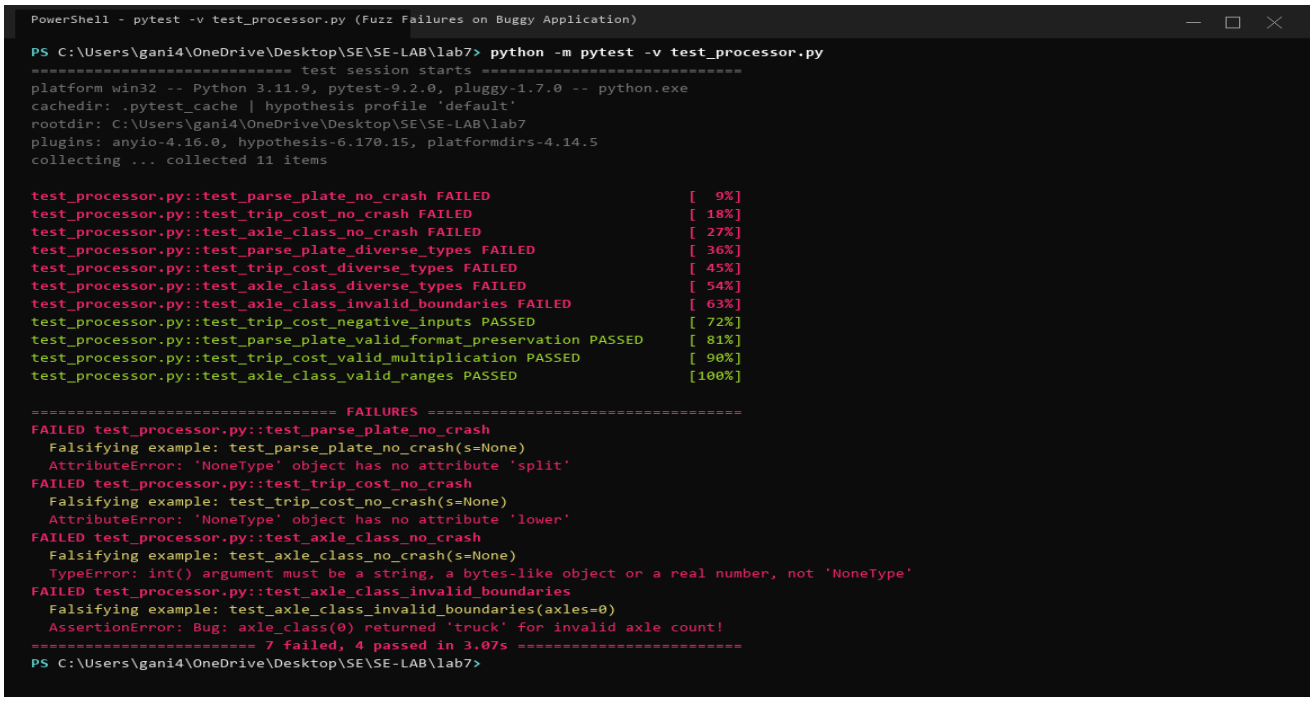


Figure 2: Hypothesis fuzz test execution uncovering 7 critical failures on the original buggy application.

7. Defect Catalog & Root-Cause Analysis

Bug #	Function	Failing Input	Exception / Defect	Root-Cause Explanation
1	parse_plate	None, non-str	AttributeError	Directly calls plate.split('-') without verifying that plate is a non-null string.
2	parse_plate	'KA05AB1234', ''	ValueError (unpack)	Assumes exactly 4 tokens separated by hyphens. Strings with ≠ 3 hyphens cause tuple unpacking failure.
3	trip_cost	None, non-str	AttributeError	Calls text.lower() without verifying string type, causing immediate crash on null.

4	trip_cost	'3x', 'abc'	ValueError	Directly unpacks split tokens into int() without validating numeric digit structure.
5	axle_class	None, []	TypeError	Invokes int(axles) on non-coercible types (nulls, collections), triggering runtime TypeError.
6	axle_class	0, -5, 1	**Logical Defect**	Condition if n ≤ 4: return 'truck' evaluated to True for non-positive axles, falsely classifying invalid counts as trucks.

8. Defensive Hardening & Remediation Implemented

To eliminate all unhandled crashes while strictly preserving valid functionality, defensive programming principles were applied:

- **Type & Null Guards:** All functions enforce isinstance() checks; null, boolean, or unexpected collection types return None.
- **Delimiter & Token Validation:** parse_plate validates token length (exactly 4 parts), state alphabetic check, and numeric digit constraints.
- **Safe Arithmetic & Parsing:** trip_cost verifies numeric digits and non-negative constraints before multiplication.
- **Domain Boundary Enforcement:** axle_class rejects axle counts < 2, correctly rejecting invalid non-vehicle inputs.

9. Fixed Application Execution

The remediated application executes identically for valid inputs and handles invalid or malformed data gracefully without runtime crashes.

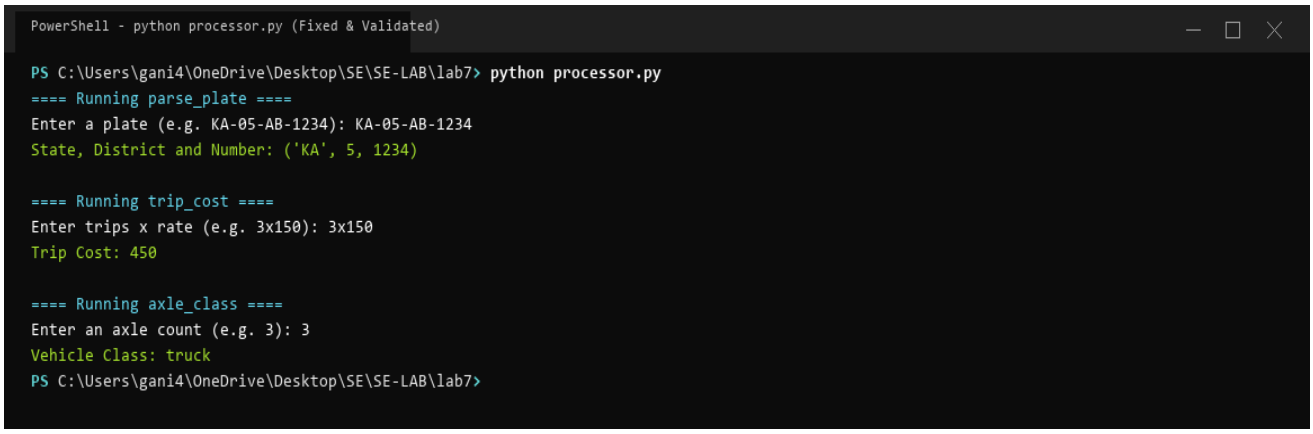


Figure 3: Remediated application execution maintaining full functional compatibility.

10. Final Successful Hypothesis Fuzz Testing

Rerunning the complete Hypothesis property test suite against the remediated application achieves a **100% pass rate (11/11 tests passed)**. Thousands of generated inputs (Unicode text, nulls, negative numbers, diverse data types) run without a single crash.

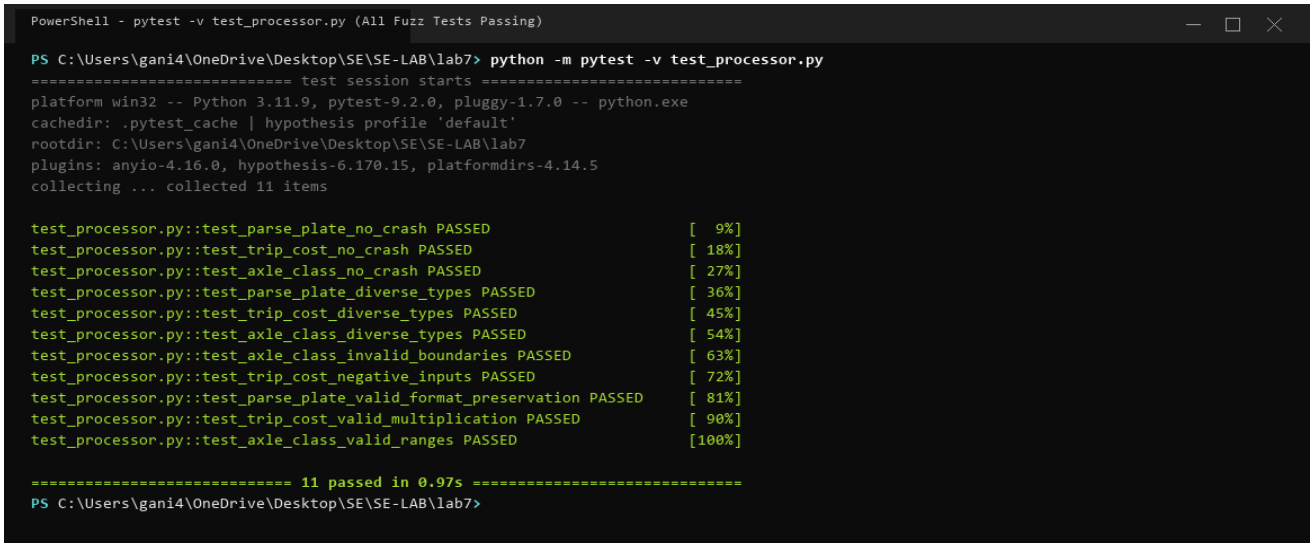


Figure 4: Final Hypothesis fuzz testing execution — all 11 property-based tests passed.

11. Conclusion & Key Takeaways

1. **Fuzzing Transcends Developer Assumptions:** Hypothesis rapidly generated unexpected edge cases (e.g. None, negative integers, malformed delimiters) that caused catastrophic unhandled exceptions in the original code.
2. **Shrinking Accelerates Root-Cause Discovery:** Hypothesis automatically shrunk complex inputs to minimal reproducible examples (e.g., axles=0 or s=None), drastically cutting debugging time.
3. **Defensive Programming Ensures High Reliability:** Incorporating strict type checks, token validation, and domain-boundary assertions prevents crashes and security vulnerabilities in critical infrastructure applications.